

线束更新并非线束收益：解耦自进化大语言模型智能体中的进化能力

Minhua Lin^{1*}, Juncheng Wu^{2*}, Zijun Wang², Zhan Shi³, Yisi Sang³, Bing He³
Zewen Liu⁴, Tianxin Wei⁵, Zongyu Wu¹, Zhiwei Zhang¹, Dakuo Wang⁶, Xiang Zhang¹
Benoit Dumoulin³, Cihang Xie², Yuyin Zhou², Suhang Wang¹, Hanqing Lu³

¹The Pennsylvania State University ²UC Santa Cruz ³Amazon

⁴Emory University ⁵UIUC ⁶Northeastern University

{mf15681, szw494}@psu.edu; {jwu418}@ucsc.edu;

{luhanqin}@amazon.com

摘要

大语言模型智能体越来越多地部署为围绕可编辑外部线束构建的系统，包括提示、技能、记忆和工具，这些线束在不改变模型参数的情况下塑造任务执行。线束自进化通过从执行证据中更新这些线束来适应此类智能体。然而，目前尚不清楚模型在任务解决中的基础能力是否能预测其在线束自进化中的能力：哪些模型能产生有用的线束更新，哪些模型实际上能从中受益？我们分析了两种线束自进化能力：（i）线束更新能力，即从执行证据中产生有用的持久线束更新的能力；（ii）线束受益能力，即在任务解决过程中从更新后的线束中受益的能力。我们的分析揭示了两项发现。首先，线束更新能力在基础能力上表现平稳：来自不同能力层级的模型产生的线束更新带来的增益惊人地相似；即使是 Qwen3.5-9B 的更新，其增益也与 Claude Opus 4.6 相当。其次，线束受益能力在基础能力上呈非单调关系：弱层级模型从更新后的线束中受益甚微，中层级模型受益最大，而强层级模型的受益程度低于中层级。我们将弱层级增益低归因于两种失败模式：弱层级模型可能无法激活相关的线束工件，或者激活了但未能忠实地遵循它们。这些发现表明，应将能力预算投入到任务解决智能体而非进化器上，并在智能体训练中针对线束调用和长程指令遵循进行优化。我们的源代码已公开，可在此处获取。

1 引言

大语言模型（LLMs）（Radford 等，2018；Touvron 等，2023）已成为语言理解（Hendrycks 等，2020）、推理（Wang * 两位作者对本文贡献相同。

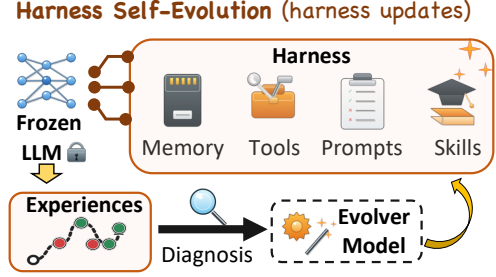


图 1：线束自进化概览。

等人，2025 年），以及任务求解（周等人，2025 年）。它们日益驱动着与外部环境交互、调用工具、操作软件接口并完成长周期任务的智能体系统（杨等人，2024b；梅里尔等人，2026 年）。在这些场景中，系统行为不仅取决于底层模型，还取决于外部智能体框架：提示（魏等人，2022 年）、技能（夏等人，2026 年）、记忆（严等人，2025 年）、工具（秦等人，2024 年）等，这些因素塑造了模型如何观察、推理、行动以及从错误中恢复。改进智能体系统日益意味着不仅要优化基础模型，还要优化其周围可编辑的框架。当前实践中，框架通常由人工设计。然而，这种人工设计在部署环境中较为脆弱：任务分布会变化，边缘情况会出现，有用的流程只有在系统与真实任务交互后才能被发现。自然的应对方式是根据执行证据自动更新框架：失败、反馈、轨迹和成功流程可以写回框架，并在未来任务中复用。我们将这种设置称为框架演化（图 1）：模型权重保持不变，而外部智能体框架随时间不断修订。近期的自演化智能体方法（马丹等人，2023 年；吴等人，2025 年；阿格拉瓦尔等人，2026 年；夏等人，2026 年；林等人，2026b）在多种场景中采用了这一思路

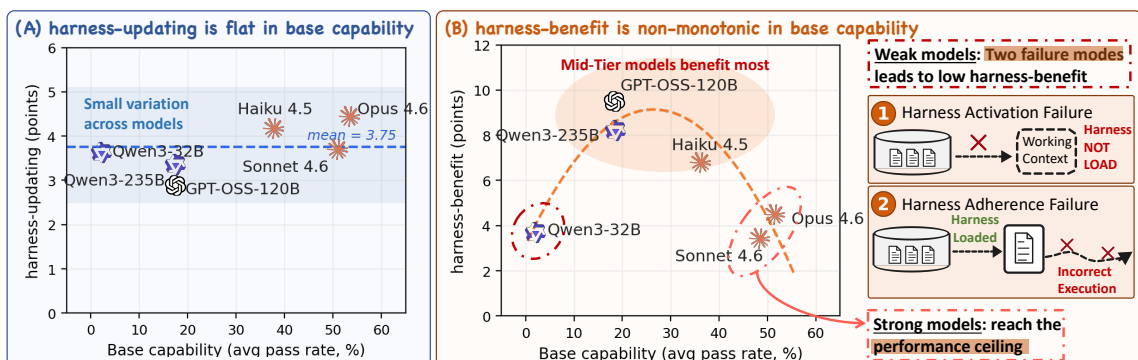


图 2：研究发现概览。 (i) 框架更新在基础能力上表现平稳。不同能力层级的模型产生的框架更新带来相似的增益。(ii) 框架收益随基础能力呈非单调变化。中等层级模型受益最大，而弱层级模型因框架激活和遵循方面的失败而受益甚微。

组件，并已展示出相对于非进化基线的端到端任务改进。在这些工作中，组件更新通常由大语言模型（LLM）根据执行证据生成；我们将这一更新角色称为进化器。尽管进展迅速，对这些方法的评估仍然只问一个端到端问题：自我进化方法能否有效提升智能体性能？这个问题很重要，但它掩盖了改进的来源。增益可能来自进化器生成更高质量的组件更新，也可能来自任务求解智能体在任务求解过程中更有效地使用更新后的组件。端到端分数无法区分这些贡献，从而留下两个悬而未决的实用问题：哪些模型能生成有用的组件更新，哪些模型能从中获益最多？为了回答这些问题，我们在三个智能体基准和七个大语言模型上，分析了模型在组件自我进化中运用的两种进化能力：组件更新能力，即从执行证据中生成有用组件更新的能力；以及组件获益能力，即在任务求解过程中从更新后的组件中获益的能力。模型作为进化器运用组件更新能力，作为任务求解智能体运用组件获益能力。我们进行了全面的实验，将七个大语言模型（涵盖开源和闭源家族、跨越不同能力层级）分别作为智能体和进化器，在三个代表性智能体基准上进行配对。我们的分析揭示了组件进化能力与基础能力之间的两个系统性解耦，其中基础能力指模型在没有组件进化时的任务求解能力（图 2）。首先，组件更新能力在基础能力上表现平稳。当我们固定任务求解智能体并改变进化器模型时，来自不同能力层级的模型

能力层级产生的工具链更新带来了惊人相似的增益，且没有任何进化器在所有基座上均占主导地位。我们的案例研究进一步表明，即使是 Qwen3.5-9B 进化器产生的工具链更新，其下游增益也能与 Claude Opus 4.6 相媲美，尽管两者基础能力差距巨大。其次，工具链收益在基础能力层级间呈非单调变化。中层级模型（如 GPT-OSS-120B）从更新后的工具链中获益最多，而强层级模型（如 Claude Opus 4.6）已达到性能上限，获益较少。然而，弱层级一端并不能用同样的上限论点来解释：在基础能力之上拥有最大提升空间的模型（如 Qwen3-32B）本应获益最多，但实际获益最少。我们的深入分析揭示了两种解释弱层级差距的失效模式：（一）工具链激活失效：弱模型在任务求解过程中常常无法调用相关工具链工件（如技能）；（二）工具链遵循失效：即使工具链被加载，弱模型也因在长程任务中指令遵循能力不足而无法遵循它。这些发现转化为工具链自进化系统的设计指导。（一）将能力预算分配给任务求解智能体，而非进化器：在任何基准上，不同进化器之间的工具链更新差距最多为 3.1 个百分点，因此扩大进化器规模收益有限；进化后的性能随任务求解智能体的变化远大于随进化器的变化。（二）将工具链调用融入智能体训练：弱层级模型常常完全无法加载工具链（例如，Qwen3-32B 的加载率为 25%，而强模型为 $\approx 96\%$ ），因此工具链调用应被视为一项需要学习的一等技能。（三）加强

长程指令遵循：即使工具链被加载，弱层级模型在轨迹中的遵循度衰减速度比强模型陡峭四倍以上，这使得持续指令遵循成为下游智能体训练的第二个关键目标。

2 相关工作

工具链工程。大语言模型智能体将冻结的主干网络与外部框架相结合，该框架协调推理、工具使用、记忆访问和环境交互（Yao 等, 2022; Yang 等, 2024b; Ning 等, 2026）。近期工作将框架视为一等设计对象，主要区别在于暴露给智能体的工件类型。提示和指令提供自然语言指导（Zhou 等, 2022; Pan 等, 2026）；工具暴露外部服务并定义智能体如何发现、调用和验证它们（Hou 等, 2025; Qin 等, 2024; Liu 等, 2025; Lin 等, 2026a）；记忆存储先前的观察、事实和策略以供后续检索（Ouyang 等, 2025; Xu 等, 2026; Fang 等, 2026）；技能将可复用过程打包为可调用模块（Li 等, 2026b; Liu 等, 2026）；代码则将框架本身视为可由智能体提议者优化的可执行源代码（Lee 等, 2026）。这些工作确立了框架作为可编辑智能体状态的地位。本文的工作将焦点从框架表示转向模型在更新和受益于框架方面的能力。更多细节见附录 A.1。

大语言模型智能体的自我进化。除了框架包含的内容之外，另一条互补的研究路线探讨如何从执行经验中更新框架。早期系统通过回合级或任务级语言反馈来适应智能体：言语自我反思（Shinn 等, 2023）和迭代自我反馈（Madaan 等, 2023）通过将经验教训反馈到上下文来改进后续尝试。更近期的方法将持久性框架组件作为自我进化的单元，从执行迹中更新提示（Agarwal 等, 2024; Zhang 等, 2025b; Agrawal 等, 2026）、记忆（Wu 等, 2025; Zhang 等, 2025a; Lin 等, 2026c）、技能（Xia 等, 2026; Alzubi 等, 2026; Yang 等, 2026）或工具（Chen 等, 2025; Li 等, 2026a）。总体而言，这些方法表明将执行经验写回框架可以提升下游任务性能。然而，该领域的评估通常报告单一更新过程与单一目标智能体在单一基座上配对后的端到端增益（Li 等, 2026b; Jiang

等人, 2026; 魏等人, 2025）。此类分数混淆了三种改进来源：智能体的基础能力、进化器的测试框架更新以及智能体的测试框架收益。本文的工作通过一项受控分析对这些方法进行了补充，该分析独立地改变任务求解智能体和进化器，分别测量测试框架更新和测试框架收益，并检验其中任一是否与基础能力相关。更多细节见附录 A.2。

3 框架进化能力

为探索框架自进化中的进化能力，本文考虑框架自进化，即在任务执行过程中通过更新固定模型周围的外部框架来适配大语言模型智能体：智能体尝试一系列任务，框架根据智能体的执行证据进行更新。本节形式化框架进化协议，并定义两种进化能力：框架更新，即产生有用框架更新的能力；框架收益，即从更新后的框架中获益的能力。

3.1 预备知识：框架状态与进化器

智能体框架。本文使用智能体框架表示大语言模型在任务执行中部署所依赖的外部非参数上下文和基础设施（Yao et al., 2022; Ning et al., 2026; Lee et al., 2026）。形式上，在进化步骤 t ，大语言模型智能体定义为：

$$A_t = (f, H_t), \quad (1)$$

其中 f 是智能体的模型骨干， H_t 是步骤 t 后的框架状态。遵循常见的框架自进化设置（Zhou et al., 2026; Lin et al., 2026b），本文保持 f 固定，仅更新 H_t 的可编辑组件（如提示、技能、记忆），并固定其他组件，如工具接口和执行策略。

进化器。进化器是将智能体的执行证据转化为框架更新的更新程序，其中近期的自进化智能体系统（Yang et al., 2024a; Yuksekgonul et al., 2024; Xia et al., 2026; Agrawal et al., 2026）越来越多地用大语言模型智能体来实例化这一程序。形式上，给定上一步的框架 H_{t-1} 和在第 t 步累积的执行证据 $\mathcal{D}_t$ ，进化器 e 提出一个框架更新并将其应用到 H_{t-1} 上，以获得下一个框架：

$$\begin{aligned} \Delta H_t &= e(H_{t-1}, \mathcal{D}_t), \\ H_t &= \text{Apply}(H_{t-1}, \Delta H_t). \end{aligned} \quad (2)$$

其中应用表示提交操作，将 ΔH_t 应用到 H_{t-1} 上。

3.2 进化协议

遵循常见的框架自进化流程 (Ouyang et al., 2025; Agrawal et al., 2026)，我们将该协议形式化为任务求解与框架进化之间的迭代循环。从初始框架 H_0 开始，该协议迭代 T 步。在每一步中，智能体在一批任务上运行，收集执行证据，进化器为下一步更新框架。

形式上，给定一个智能体 $A_{t-1} = (f, H_{t-1})$ 以及在第 t , A_{t-1} 步的任务批次 $\mathcal{X}_t$ ，智能体尝试求解每个任务 $x \in \mathcal{X}_t$ 并输出：

$$(\tau_{t,x}, y_{t,x}) = \text{Solve}(A_{t-1}, x) \quad (3)$$

其中 $\tau_{t,x}$ 是执行轨迹， $y_{t,x}$ 是最终输出。执行证据 $\mathcal{D}_t$ 则为：

$$\mathcal{D}_t = \{(x, \tau_{t,x}, y_{t,x}) : x \in \mathcal{X}_t\}. \quad (4)$$

进化器从 H_{t-1} 和 $\mathcal{D}_t$ as in Eq. 2 中产生更新后的框架 H_t ，从而得到下一个智能体 $A_t = (f, H_t)$ 。该循环重复 T 步，产生最终框架 H_T 。

3.3 能力度量

为了分析哪些模型能够产生有用的框架更新以及哪些模型能够从中受益，我们正式定义了三个度量来衡量框架演化能力（即框架更新能力和框架受益能力）以及每个模型的基础能力。基础能力与演化增益。给定任务集 $\mathcal{X} = \bigcup_{t=1}^T \mathcal{X}_t$ ，模型 f 的基础能力是初始智能体 $A_0 = (f, H_0)$ 在 $\mathcal{X}$ 上的任务解决性能：

$$M_{\text{base}}(f) = J_{\mathcal{X}}(f, H_0), \quad (5)$$

其中 $J_{\mathcal{X}}(f, H)$ 是评分函数，用于衡量智能体 (f, H) 在 $\mathcal{X}$ 上的性能。给定模型 f 和演化器 e ，令 $H_T^{(f,e)}$ 表示以 f 为智能体、以 e 为演化器，从 H_0 开始经过 T 步演化后产生的最终框架。我们进一步将成对演化增益定义为特定智能体-演化器配对 (f, e) 相对于演化前智能体任务解决性能的提升：

$$\Delta(f, e) = J_{\mathcal{X}}(f, H_T^{(f,e)}) - M_{\text{base}}(f). \quad (6)$$

框架更新能力。演化器 e 的框架更新能力是指其产生能够提升智能体任务解决能力的框架更新的能力。形式上，这被定义为在锚定智能体集合 $\mathcal{F}^*$ 上的平均成对增益：

$$\Delta_{\text{update}}(e) = \frac{1}{|\mathcal{F}^*|} \sum_{f \in \mathcal{F}^*} \Delta(f, e). \quad (7)$$

框架受益能力。模型 f 的框架受益能力是指其通过框架自演化在任务解决性能上获得的最大增益。在实践中，我们将其估计为在固定锚定演化器集合 $\mathcal{E}^*$ 上的最大成对增益：

$$\Delta_{\text{benefit}}(f) = \max_{e \in \mathcal{E}^*} \Delta(f, e). \quad (8)$$

4 实验

在本节中，我们实证分析第 3 节中定义的两测试框架演化能力。我们在第 4.2 节中给出测试框架更新能力的演化侧分析，并在第 4.3 节中给出测试框架受益能力的智能体侧分析。

4.1 实验设置

数据集。我们在三个具有代表性的智能体基准上评估：用于软件工程的 SWE-bench Verified (SWE) (Jimenez 等, 2024)、用于在真实 MCP 服务器上进行工具使用的 MCP-Atlas (MCP) (Bandi 等, 2026)，以及用于跨多个领域基于技能执行的 Skills-Bench (SB) (Li 等, 2026b)。这些数据集的更多细节见附录 B.1。

模型。我们使用七个大语言模型骨干，涵盖开源和闭源系列，跨越不同能力层级。对于智能体侧分析，我们使用六个模型：Claude Opus 4.6 (Anthropic, 2026a)、Claude Sonnet 4.6 (Anthropic, 2026b)、Claude Haiku 4.5 (Anthropic, 2025)、Qwen3-235B-A22B 和 Qwen3-32B (Yang 等, 2025)，以及 GPT-OSS-120B (Agarwal 等, 2025)。对于演化侧分析，我们使用相同的六个模型加上 Qwen3.5-9B (Qwen, 2026)，这是本文中最小的模型，用于测试一个显著更小的开源模型是否仍能产生有用的测试框架更新。

评估协议。我们报告第 3.3 节中定义的三个度量：基础能力 $M_{\text{base}}(f)$ 、测试框架更新增益 $\Delta_{\text{update}}(e)$ ，以及测试框架受益增益 $\Delta_{\text{benefit}}(f)$ 。为了计算它们，我们使用通过率作为三个基准上 $J_{\mathcal{X}}$ 的主要度量。我们考虑一种原位评估设置： $\mathcal{X}_t$ 中的每个任务在 H_{t-1} 之前被评分。

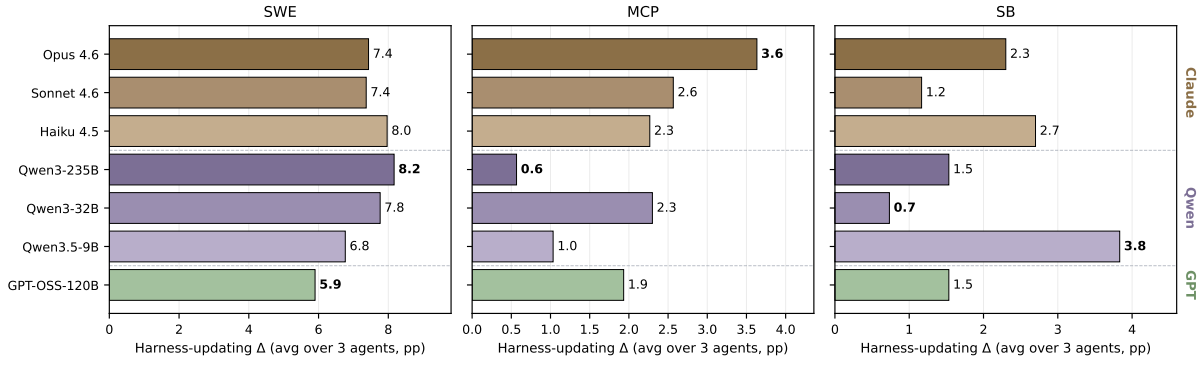


图 3: 各进化器的线束更新能力 (Δ_{update})。进化器按模型家族 (Claude、Qwen、GPT-OSS) 分组。每个面板内以粗体标记的最佳和最差进化器随基准变化。

其证据用于生成 H_t 。最终结果通过聚合任务流上的每任务得分来报告。更多细节见附录 B.3。

实现细节。我们以固定的求解-进化循环实例化第 3.2 节中的进化协议。为公平比较，我们为所有智能体-进化器对固定了智能体和进化器的提示模板以及轨迹窗口；仅大语言模型骨干不同。每个基准内的所有对从相同的初始线束 H_0 和任务流 $\mathcal{X}$ 开始，共享相同的进化预算 β 和每任务轮次限制。可进化组件为 SWE-bench Verified 和 SkillsBench 的技能，以及 MCP-Atlas 的技能、提示和记忆。其他细节如提示模板见附录 B.4。

4.2 进化器侧分析

为理解线束更新能力在不同大语言模型间的差异，我们固定任务求解智能体，并在第 4.1 节的七种大语言模型上变化进化器。具体而言，我们使用三种代表性大语言模型 Opus 4.6、Sonnet 4.6 和 Qwen3-235B 作为 $\mathcal{F}^*$ 中的锚定智能体。对于每个进化器 e ，我们在图 3 中报告第 3.3 节定义的 $\Delta_{\text{update}}(e)$ 在三个基准上的结果。所有智能体-进化器配对的完整通过率结果见附录 C.1。观察 1: 基础能力中的线束更新表现平稳。

能力。图 3 显示两种模式：(i) 进化器间 Δ_{update} 的分布较窄。在任何基准上，最佳与最差进化器之间的差距至多为 3.1 个百分点，且没有模型在所有基准上获胜。Qwen3-235B 体现了这种重新排序：它在 SWE 上领先 (8.2 个百分点)，但在 MCP 上排名最后 (0.6 个百分点)。(ii) 模型规模不具有预测性。最小的进化器 Qwen3.5-9B 在 SB 上获得最高增益 (3.8 个百分点)，超过

Opus 4.6 (2.3 个百分点) 和 Qwen3-235B (1.5 个百分点) 均有所提升。

案例研究：9B 进化器编写的技能与 Opus 的技能在程序上同构。为理解这些可比增益背后的机制，我们详细考察了一个具有代表性的 SkillsBench 任务 flink-query。我们将任务求解代理的主干固定为 Opus 4.6，并比较其在三种进化器条件下的轨迹 (图 4)：无进化器、Qwen3.5-9B 作为进化器、Opus 4.6 作为进化器。我们观察到，在没有进化技能的情况下，代理遗漏了 FINISH 事件过滤器，未能解决该任务 (得分 0.67)；在注入由 Qwen3.5-9B 或 Opus 4.6 提供的技能后，同一代理成功解决了任务 (得分 1.0)。检查这两种技能后，我们发现它们在程序上是同构的，规定了相同的步骤序列，仅在实现细节和冗长程度上有所不同。因此，9B 开源进化器达到了与前沿进化器相同的程序内容。技能内容和分析的完整细节见附录 C.2。

观察 2: 进化后的得分主要由模型的基础能力决定，而非进化器的身份。为理解任务求解代理和进化器对进化后性能的相对贡献，我们在 $\mathcal{F}^*$ 中绘制了三个大语言模型 (Opus 4.6、Sonnet 4.6、Qwen3-235B) 在 Sec. 4.1 中七个大语言模型作为进化器更新的测试框架下的任务求解性能，并与每个代理的基础能力进行对比。MCP-Atlas 上的结果如图 5 所示。我们观察到：(i) 代理内部的差异远小于代理之间的差距。七个进化器在代理内部的差异最大为 5.1 个百分点 (Qwen3-235B)，相对于 Opus 和 Qwen3-235B 基础能力之间 36.0 个百分点的差距而言很小。这一模式在 SWE 和 SB 上持续存在。(ii) 极端配对仍然有利于强代理。

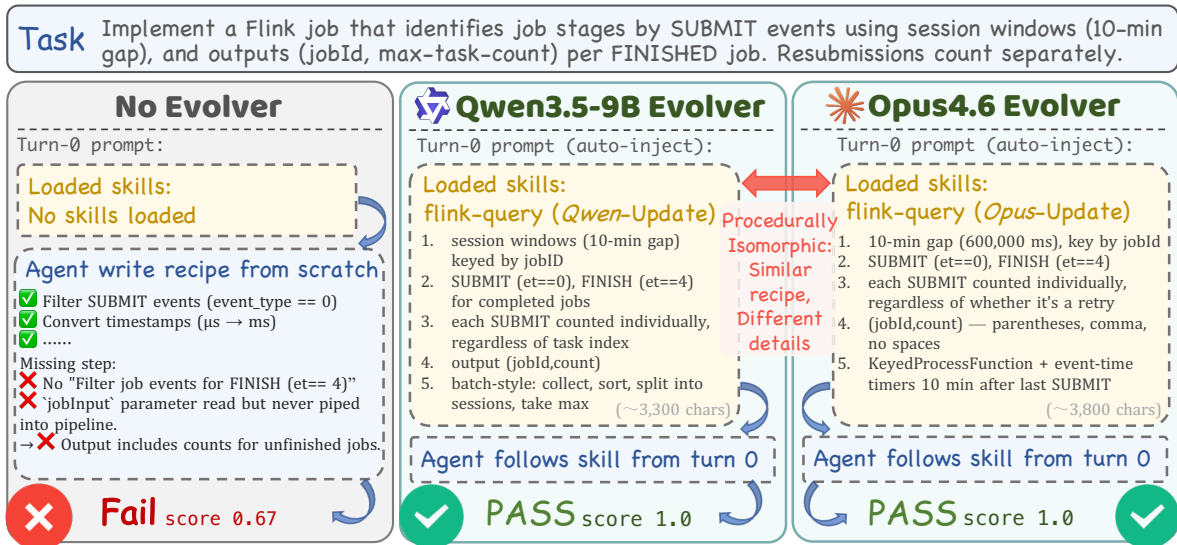


图 4: Qwen3.5-9B 和 Claude Opus 4.6 更新的测试框架比较。我们在三种条件下比较了 Opus 4.6 代理在 SkillsBench flink-query 任务上的表现：无进化技能（左，得分 0.67）、由 Qwen3.5-9B 进化的技能（中，得分 1.0）、由 Opus 4.6 进化的技能（右，得分 1.0）。两种进化技能都编码了程序上相似的指导，并使同一代理能够解决该任务。

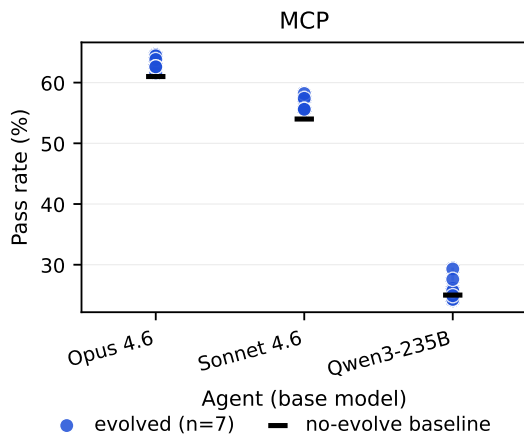


图 5: MCP 进化后得分：对于每个锚定代理，每个蓝点代表七个进化得分之一，黑色刻度线为无进化基线。代理内部跨进化器的变异相对于代理间基础能力的变异较小。

即使将最弱的锚定智能体与其表现最佳的进化器配对，对抗最强的锚定智能体与其表现最差的进化器，强智能体在每个基准上仍领先 18.6 至 35.2 个百分点。这两种模式在 SWE 和 SB 数据集上也持续存在（附录 C.3）。因此，进化后的性能瓶颈在于智能体侧，而非进化器侧，这促使了第 4.3 节中的智能体侧分析。

要点。将能力预算分配给任务解决智能体，而非进化器：(i) Δ_{update} 在任何基准上跨进化器的变化最多为 3.1 个百分点，

且 (ii) 进化后的得分主要由智能体的基础能力决定。

4.3 智能体侧分析

为了理解不同大语言模型 (LLM) 中工具增强能力的变化，我们固定进化器，并在第 4.1 节中的大语言模型骨干上改变任务解决智能体：Opus 4.6、Sonnet 4.6、Haiku 4.5、Qwen3-235B、Qwen3-32B 和 GPT-OSS-120B。我们使用 Opus 4.6、Sonnet 4.6 和 Qwen3-235B 作为三个锚定进化器，记为 $\mathcal{E}^*$ 。对于每个智能体 f ，我们在表 1 和图 6 中报告第 3.3 节定义的 $\Delta_{\text{benefit}}(f)$ 。所有智能体-进化器配对的完整通过率结果见附录 D.2 中的表 7。

观察 1: Δ_{benefit} 是非单调的，在

基础能力。如表 1 和图 6 所示， Δ_{benefit} 并未随基础能力单调递增。在 SWE 上，增益在 Qwen3-235B 处达到峰值 (19.3 个百分点)，而较弱的基础模型 Qwen3-32B 仅获得 4.4 个百分点的增益，较强的 Opus 4.6 也仅获得 2.6 个百分点的增益。在 MCP 上，峰值移至 GPT-OSS-120B (7.0 个百分点)，同样在基础能力标尺的两端增益较低。这一模式在能力标尺的两端有不同的解释。在高能力端，较小的增益与天花板效应一致：强模型在初始测试框架下已能解决许多任务，留给进一步改进的空间较小。然而，在低能力端，较小的增益反映了不同的瓶颈，我们将在下文中诊断。观察 2: 弱层级模型获得的

表 1：各基准上的基础通过率 (%) 与测试框架增益 $\Delta\text{benefit}$ (百分点)。每一行对应一个用作任务求解智能体的大语言模型骨干。粗体标记每个基准内最大的 $\Delta\text{benefit}$ 。

Model	SWE		MCP		SB	
	Base	Δ	Base	Δ	Base	Δ
Qwen3-32B	3.6	4.4	3.6	1.0	0.0	5.8
Qwen3-235B	20.7	19.3	25.0	4.3	4.7	1.1
GPT-OSS-120B	26.2	15.8	28.0	7.0	0.0	7.0
Haiku 4.5	66.0	2.4	42.4	3.6	5.8	15.1
Sonnet 4.6	73.2	2.8	54.0	3.2	24.4	3.5
Opus 4.6	74.2	2.6	61.0	3.6	25.6	5.8

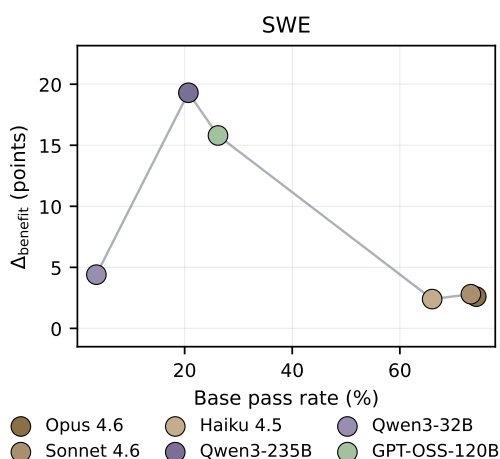


图 6：SWE 上 $\Delta\text{benefit}$ 与基础通过率的关系。每个点对应一个用作任务求解智能体的大语言模型骨干；点按基础通过率升序连接。MCP 和 SB 的对应图见附录 D.2。

$\Delta\text{benefit}$ 源于两种失效模式。为理解基础能力较低的弱层级模型为何获得较低的 $\Delta\text{benefit}$ ，我们在 SkillsBench 上进行了深入分析，识别出两种互补的失效模式：测试框架激活与测试框架遵循，如图 7 所示。第一种模式是测试框架激活失败：弱层级模型往往未能将相关测试框架工件（如技能）纳入其工作上下文。为在 SkillsBench 上量化这一点，我们报告每个智能体的技能加载率（SLR），即其轨迹中主动将至少一项技能加载到上下文中的比例。表 2 显示，Opus 4.6、Sonnet 4.6 和 Qwen3-235B 的技能加载率接近上限（0.957 - 0.961），但 GPT-OSS-120B 降至 0.446，Qwen3-32B 降至 0.251。图 7 左图展示了这种激活失败。具体而言，Qwen3-32B 识别出了相关技能，但将加载请求嵌入到更广泛的动作中，而非作为独立的技能加载动作发出。SkillsBench

表 2：SkillsBench 上各模型的激活率、遵循率与结果度量。SLR：模型轨迹中至少加载一项技能到上下文的比例。HFR：已加载技能的轨迹中被判定为遵循所加载技能指导的比例。LPR：模型已加载技能轨迹中的通过率。模型按 SkillsBench 基础能力排序。

Model	SLR	HFR	LPR
Qwen3-32B	0.251	0.142	0.023
GPT-OSS-120B	0.446	0.442	0.040
Haiku 4.5	0.794	0.600	0.099
Qwen3-235B	0.961	0.350	0.022
Sonnet 4.6	0.959	0.730	0.145
Opus 4.6	0.957	0.757	0.177

环境因此不将其视为有效的加载请求，技能主体从未进入上下文。第二种模式是框架遵循失败：即使相关框架工件已加载，弱层级模型在任务求解过程中也常常无法忠实遵循其指导。我们用框架遵循率（HFR）量化这一失败，该指标在至少加载了一项技能的轨迹上计算。对于每条已加载技能的任务求解轨迹，由大语言模型评判者判定任务求解模型是否遵循了所加载技能的指导。HFR 即被判定为遵循技能的已加载技能轨迹所占比例。附录 D.3 提供了评判流程的细节。表 2 报告了 HFR 以及两个互补度量：SLR 衡量框架激活，加载后通过率（LPR）衡量该模型已加载技能轨迹中的通过率。我们观察到两种模式。（一）强层级模型的框架遵循率远高于弱层级模型。Opus 4.6 的 HFR 达到 0.757，而 Qwen3-32B 仅为 0.142。（二）加载框架并不足以从中获益。Qwen3-235B 最清晰地分离了激活与遵循：其技能加载率为 0.961，几乎与 Opus 4.6 相同，但其 HFR 仅为 0.350。其加载后通过率也反映了这一差距，为 0.022，而 Opus 4.6 为 0.177。图 7 右面板中的 **pg-essay-to-audiobook** 案例说明了这种遵循失败。Qwen3-32B 成功加载了程序性技能，但将指导视为现成脚本而非需要遵循的程序。在第一次尝试失败后，它终止了任务，而不是尝试技能规定的替代步骤。更多分析细节见附录 D.1。诊断：长上下文下的弱指令遵循——

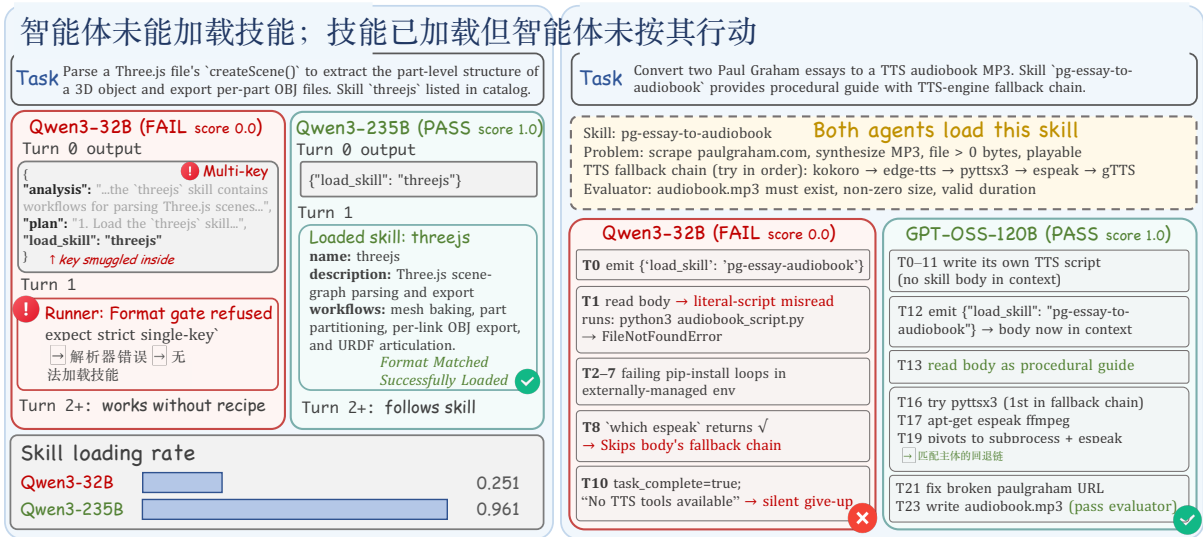


图 7: Qwen3-32B 在 SkillsBench 上的两种框架收益失效模式。左图 (threejs)：框架激活失败，无效的多键加载操作阻止技能主体进入上下文。右图 (pg-essay-to-audiobook)：框架遵循失败，技能已加载但智能体将其视为字面脚本，跳过了规定的回退链。

横向执行。为检验框架遵循度是否随轨迹展开而下降，我们进行了阶段级遵循度分析。由大语言模型评判器在不同执行阶段给出 0 – 1 的遵循度评分，细节见附录 D.4。我们分别使用 Qwen3-32B、GPT-OSS-120B 和 Opus 4.6 作为弱、中、强三个层级的代表模型。表 3 报告了三个代表性阶段：框架加载后、轨迹中点和最终验证，评分为各模型被评判轨迹的平均值。我们观察到 Qwen3-32B 从框架加载后的 0.52 急剧下降至最终验证时的 0.13，而 GPT-OSS-120B 从 0.67 较为平缓地降至 0.43。相比之下，Opus 4.6 保持稳定，从 0.89 到 0.80。这种分级漂移表明存在长程指令遵循瓶颈：较弱模型在轨迹展开过程中逐渐丧失遵循度，而非仅在加载时误读框架。

要点。智能体训练应针对框架收益的两个维度进行。(i) 将框架调用融入训练：弱层级模型的技能加载率较低 (Qwen3-32B 为 25.1%，而强层级模型为 $\approx 96\%$)，因此智能体必须学会可靠地将相关框架工件引入上下文。(ii) 加强长程指令遵循：即使在加载框架后，弱层级模型在轨迹过程中也会丧失遵循度 (Qwen3-32B 从 0.52 漂移至 0.13)，因此智能体必须学会在长程任务中持续维持框架指导。

表 3: 代表性弱、中、强层级模型 (Qwen3-32B、GPT-OSS-120B 和 Opus 4.6) 的分阶段遵循度评分。粗体和下划线分别标记每个阶段的最佳和最差评分。

Trajectory Phase	Qwen3-32B (weak)	GPT-OSS (mid)	Opus 4.6 (strong)
Harness loaded	0.52	0.67	0.89
Mid turn	0.22	0.48	0.79
Final turn	0.13	0.43	0.80
drift (load → final)	-0.39	-0.24	-0.09

5 结论

我们通过将线束自进化分解为两种区别于基础能力的模型能力来分析它：线束更新能力，即产生线束更新的能力；以及线束受益能力，即在任务解决过程中从更新后的线束中获益的能力。在七个大语言模型和三个基准上，线束更新能力在基础能力上表现平稳：不同能力层级的模型产生的更新带来相似的增益，甚至 Qwen3.5-9B 进化器带来的增益与 Claude Opus 4.6 相当。相比之下，线束受益能力在基础能力上呈非单调变化：弱能力层级的模型获益甚微，这归因于两种失败模式：未能激活相关的线束工件，以及一旦激活后未能忠实地遵循它们。这些发现促使我们将能力预算投入到智能体而非进化器上，并针对线束调用和长程指令遵循来训练智能体。

6 局限性

我们的研究聚焦于线束自进化，其中模型权重保持不变，适应通过对外部线束工件的更新来实现。我们没有评估参数微调、模型权重的强化学习，或将权重更新与线束更新相结合的混合适应方法。我们的模型集具有代表性但并非详尽无遗：我们纳入了跨多个能力层级的开源和闭源模型，但更广泛的模型网格将进一步阐明线束更新能力和线束受益能力如何随模型家族、规模、训练方案和部署成本而变化。

7 伦理声明

这项工作研究了大语言模型智能体，它们根据执行证据更新持久的外部线束。所有实验均在基准任务上进行，我们不收集或处理私人用户数据。然而，线束自进化引发了更广泛的部署问题，因为更新后的线束可能会在未来的任务中持续存在。错误的经验、不安全的工具使用规则、有偏见的指令或敏感信息可能被写入线束并被后续智能体重用。在我们的评估中，线束更新被记录，进化器被限制不得修改评估脚本或更新模型权重。这些控制使基准设置可审计，但它们本身并不能保证开放部署中的安全性。现实世界中的线束自进化系统应将隐私、数据保留的同意、更新可逆性、可审计性和人工监督视为首要设计需求。

参考文献

- Eshaan Agarwal, Joykirat Singh, Vivek Dani, Raghav Magazine, Tanuja Ganu, and Akshay Nambi. 2024. Promptwizard: Task-aware prompt optimization framework. *arXiv preprint arXiv:2405.18369*.
- Sandhini Agarwal, Lama Ahmad, Jason Ai, Sam Altman, Andy Applebaum, Edwin Arbus, Rahul K Arora, Yu Bai, Bowen Baker, Haiming Bao, et al. 2025. gpt-oss-120b & gpt-oss-20b model card. *arXiv preprint arXiv:2508.10925*.
- Lakshya A Agrawal, Shangyin Tan, Dilara Soylu, Noah Ziemis, Rishi Khare, Krista Opsahl-Ong, Arnav Singhvi, Herumb Shandilya, Michael J Ryan, Meng Jiang, Christopher Potts, Koushik Sen, Alex Dimakis, Ion Stoica, Dan Klein, Matei Zaharia, and Omar Khattab. 2026. GEPA: Reflective prompt evolution can outperform reinforcement learning. In *The Fourteenth International Conference on Learning Representations*.
- Salaheddin Alzubi, Noah Provenzano, Jaydon Bingham, Weiyuan Chen, and Tu Vu. 2026. Evoskill: Automated skill discovery for multi-agent systems. *arXiv preprint arXiv:2603.02766*.
- Anthropic. 2025. Claude haiku 4.5 system card.
- Anthropic. 2026a. Claude opus 4.6 system card.
- Anthropic. 2026b. Claude sonnet 4.6 system card.
- Chaithanya Bandi, Ben Hertzberg, Geobio Boo, Tejas Polakam, Jeff Da, Sami Hassaan, Manasi Sharma, Andrew Park, Ernesto Hernandez, Dan Rambado, et al. 2026. Mcp-atlas: A large-scale benchmark for tool-use competency with real mcp servers. *arXiv preprint arXiv:2602.00933*.
- Guoxin Chen, Zhong Zhang, Xin Cong, Fangda Guo, Yesai Wu, Yankai Lin, Wenzheng Feng, and Yasheng Wang. 2025. Learning evolving tools for large language models. In *The Thirteenth International Conference on Learning Representations*.
- Jizhan Fang, Xinle Deng, Haoming Xu, Ziyang Jiang, Yuqi Tang, Ziwen Xu, Shumin Deng, Yunzhi Yao, Mengru Wang, Shuofei Qiao, Huajun Chen, and Ningyu Zhang. 2026. Lightmem: Lightweight and efficient memory-augmented generation. In *The Fourteenth International Conference on Learning Representations*.
- Dan Hendrycks, Collin Burns, Steven Basart, Andy Zou, Mantas Mazeika, Dawn Song, and Jacob Steinhardt. 2020. Measuring massive multitask language understanding. *arXiv preprint arXiv:2009.03300*.
- Xinyi Hou, Yanjie Zhao, Shenao Wang, and Haoyu Wang. 2025. Model context protocol (mcp): Landscape, security threats, and future research directions. *ACM Transactions on Software Engineering and Methodology*.
- Sihang Jiang, Lipeng Ma, Zhonghua Hong, Keyi Wang, Zhiyu Lu, Shisong Chen, Jinghao Zhang, Tianjun Pan, Weijia Zhou, Jiaqing Liang, et al. 2026. Sea-eval: A benchmark for evaluating self-evolving agents beyond episodic assessment. *arXiv preprint arXiv:2604.08988*.
- Carlos E Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. 2024. Swe-bench: Can language models resolve real-world github issues? In *International Conference on Learning Representations*, volume 2024, pages 54107–54157.
- Yoonho Lee, Roshen Nair, Qizheng Zhang, Kangwook Lee, Omar Khattab, and Chelsea Finn. 2026. Meta-harness: End-to-end optimization of model harnesses. *arXiv preprint arXiv:2603.28052*.

- Haotian Li, Shijun Yang, Weizhen Qi, Silei Zhao, Rui Hua, Mingzhu Song, Xiaojian Yang, and Chao Peng. 2026a. Yunjue agent tech report: A fully reproducible, zero-start in-situ self-evolving agent system for open-ended tasks. *arXiv preprint arXiv:2601.18226*.
- Xiangyi Li, Wenbo Chen, Yimin Liu, Shenghan Zheng, Xiaokun Chen, Yifeng He, Yubo Li, Bingran You, Haotian Shen, Jiankai Sun, et al. 2026b. Skillsbench: Benchmarking how well agent skills work across diverse tasks. *arXiv preprint arXiv:2602.12670*.
- Minhua Lin, Enyan Dai, Hui Liu, Xianfeng Tang, Yuliang Yan, Zhenwei Dai, Jingying Zeng, Zhiwei Zhang, Fali Wang, Hongcheng Gao, Chen Luo, Xiang Zhang, Qi He, and Suhang Wang. 2026a. [How far are LLMs from professional poker players? revisiting game-theoretic reasoning with agentic tool use](#). In *The Fourteenth International Conference on Learning Representations*.
- Minhua Lin, Hanqing Lu, Zhan Shi, Bing He, Rui Mao, Zhiwei Zhang, Zongyu Wu, Xianfeng Tang, Hui Liu, Zhenwei Dai, Rui Zhang, Xiang Zhang, Suhang Wang, Benoit Dumoulin, and Jian Pei. 2026b. [Position: Agentic evolution is the path to evolving LLMs](#). In *First Workshop on Agent Skills*.
- Minhua Lin, Zongyu Wu, Zhichao Xu, Hui Liu, Xianfeng Tang, Qi He, Charu Aggarwal, Xiang Zhang, and Suhang Wang. 2025. A comprehensive survey on reinforcement learning-based agentic search: Foundations, roles, optimizations, evaluations, and applications. *arXiv preprint arXiv:2510.16724*.
- Minhua Lin, Zhiwei Zhang, Hanqing Lu, Hui Liu, Xianfeng Tang, Qi He, Xiang Zhang, and Suhang Wang. 2026c. Memma: Coordinating the memory cycle through multi-agent reasoning and in-situ self-evolution. *arXiv preprint arXiv:2603.18718*.
- Dawei Liu, Zongxia Li, Hongyang Du, Xiyang Wu, Shihang Gui, Yongbei Kuang, and Lichao Sun. 2026. Graph of skills: Dependency-aware structural retrieval for massive agent skills. *arXiv preprint arXiv:2604.05333*.
- Weiwen Liu, Xu Huang, Xingshan Zeng, Shuai Yu, Dexun Li, Shuai Wang, Weinan Gan, Zhengying Liu, Yuanqing Yu, Zezhong WANG, et al. 2025. Toolace: Winning the points of llm function calling. In *International Conference on Learning Representations*, volume 2025, pages 41359–41381.
- Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegrefe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, et al. 2023. Self-refine: Iterative refinement with self-feedback. *Advances in neural information processing systems*, 36:46534–46594.
- Mike A Merrill, Alexander G Shaw, Nicholas Carlini, Boxuan Li, Harsh Raj, Ivan Bercovich, Lin Shi, Jeong Yeon Shin, Thomas Walshe, E Kelly Buchanan, et al. 2026. Terminal-bench: Benchmarking agents on hard, realistic tasks in command line interfaces. *arXiv preprint arXiv:2601.11868*.
- Xuying Ning, Katherine Tieu, Dongqi Fu, Tianxin Wei, Zihao Li, Yuanchen Bei, Jiaru Zou, Mengting Ai, Zhining Liu, Ting-Wei Li, et al. 2026. Code as agent harness. *arXiv preprint arXiv:2605.18747*.
- Siru Ouyang, Jun Yan, I Hsu, Yanfei Chen, Ke Jiang, Zifeng Wang, Rujun Han, Long T Le, Samira Daruki, Xiangru Tang, et al. 2025. Reasoningbank: Scaling agent self-evolving with reasoning memory. *arXiv preprint arXiv:2509.25140*.
- Linyue Pan, Lexiao Zou, Shuo Guo, Jingchen Ni, and Hai-Tao Zheng. 2026. Natural-language agent harnesses. *arXiv preprint arXiv:2603.25723*.
- Yujia Qin, Shihao Liang, Yining Ye, Kunlun Zhu, Lan Yan, Yaxi Lu, Yankai Lin, Xin Cong, Xiangru Tang, Bill Qian, et al. 2024. Toolllm: Facilitating large language models to master 16000+ real-world apis. In *International Conference on Learning Representations*, volume 2024, pages 9695–9717.
- Team Qwen. 2026. [Qwen3.5: Accelerating productivity with native multimodal agents](#).
- Alec Radford, Karthik Narasimhan, Tim Salimans, Ilya Sutskever, et al. 2018. Improving language understanding by generative pre-training.
- Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. 2023. Reflexion: Language agents with verbal reinforcement learning. *Advances in neural information processing systems*, 36:8634–8652.
- Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, et al. 2023. Llama: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971*.
- Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. 2023. Voyager: An open-ended embodied agent with large language models. *arXiv preprint arXiv:2305.16291*.
- Ruonan Wang, Runxi Wang, Yunwen Shen, Chengfeng Wu, Qinglin Zhou, and Rohitash Chandra. 2025. Evaluation of llms for mathematical problem solving. *arXiv preprint arXiv:2506.00309*.
- Zora Zhiruo Wang, Jiayuan Mao, Daniel Fried, and Graham Neubig. 2024. Agent workflow memory. *arXiv preprint arXiv:2409.07429*.
- Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Fei Xia, Ed Chi, Quoc V Le, Denny Zhou, et al. 2022. Chain-of-thought prompting elicits reasoning in large language models. *Advances in neural information processing systems*, 35:24824–24837.

- Tianxin Wei, Noveen Sachdeva, Benjamin Coleman, Zhankui He, Yuanchen Bei, Xuying Ning, Mengting Ai, Yunzhe Li, Jingrui He, Ed H Chi, et al. 2025. Evo-memory: Benchmarking llm agent test-time learning with self-evolving memory. *arXiv preprint arXiv:2511.20857*.
- Rong Wu, Xiaoman Wang, Jianbiao Mei, Pinlong Cai, Daocheng Fu, Cheng Yang, Licheng Wen, Xueming Yang, Yufan Shen, Yuxin Wang, et al. 2025. Evolver: Self-evolving llm agents through an experience-driven lifecycle. *arXiv preprint arXiv:2510.16079*.
- Peng Xia, Jianwen Chen, Hanyang Wang, Jiaqi Liu, Kaide Zeng, Yu Wang, Siwei Han, Yiyang Zhou, Xujia Zhao, Haifeng Chen, et al. 2026. Skillrl: Evolving agents via recursive skill-augmented reinforcement learning. *arXiv preprint arXiv:2602.08234*.
- Wujiang Xu, Zujie Liang, Kai Mei, Hang Gao, Juntao Tan, and Yongfeng Zhang. 2026. A-mem: Agentic memory for llm agents. *Advances in Neural Information Processing Systems*, 38:17577–17604.
- Sikuan Yan, Xiufeng Yang, Zuchao Huang, Ercong Nie, Zifeng Ding, Zonggen Li, Xiaowen Ma, Jinhe Bi, Kristian Kersting, Jeff Z Pan, et al. 2025. Memory-rl: Enhancing large language model agents to manage and utilize memories via reinforcement learning. *arXiv preprint arXiv:2508.19828*.
- An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, et al. 2025. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*.
- Chengrun Yang, Xuezhi Wang, Yifeng Lu, Hanxiao Liu, Quoc V Le, Denny Zhou, and Xinyun Chen. 2024a. Large language models as optimizers. In *International Conference on Learning Representations*, volume 2024, pages 12028–12068.
- John Yang, Carlos E Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. 2024b. Swe-agent: Agent-computer interfaces enable automated software engineering. *Advances in Neural Information Processing Systems*, 37:50528–50652.
- Yutao Yang, Junsong Li, Qianjun Pan, Bihao Zhan, Yuxuan Cai, Lin Du, Jie Zhou, Kai Chen, Qin Chen, Xin Li, et al. 2026. Autoskill: Experience-driven lifelong learning via skill self-evolution. *arXiv preprint arXiv:2603.01145*.
- Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. 2022. React: Synergizing reasoning and acting in language models. *arXiv preprint arXiv:2210.03629*.
- Mert Yuksekgonul, Federico Bianchi, Joseph Boen, Sheng Liu, Zhi Huang, Carlos Guestrin, and James Zou. 2024. Textgrad: Automatic "differentiation" via text. *arXiv preprint arXiv:2406.07496*.
- Guibin Zhang, Haotian Ren, Chong Zhan, Zhenhong Zhou, Junhao Wang, He Zhu, Wangchunshu Zhou, and Shuicheng Yan. 2025a. Memevolve: Meta-evolution of agent memory systems. *arXiv preprint arXiv:2512.18746*.
- Qizheng Zhang, Changran Hu, Shubhangi Upasani, Boyuan Ma, Fenglu Hong, Vamsidhar Kamanuru, Jay Rainton, Chen Wu, Mengmeng Ji, Hanchen Li, et al. 2025b. Agentic context engineering: Evolving contexts for self-improving language models. *arXiv preprint arXiv:2510.04618*.
- Andrew Zhao, Daniel Huang, Quentin Xu, Matthieu Lin, Yong-Jin Liu, and Gao Huang. 2024. Expel: Llm agents are experiential learners. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 38, pages 19632–19642.
- Chenyu Zhou, Huacan Chai, Wenteng Chen, Zihan Guo, Rong Shan, Yuanyi Song, Tianyi Xu, Yingxuan Yang, Aofan Yu, Weiming Zhang, et al. 2026. Externalization in llm agents: A unified review of memory, skills, protocols and harness engineering. *arXiv preprint arXiv:2604.08224*.
- Xiyuan Zhou, Xinlei Wang, Yirui He, Yang Wu, Ruixi Zou, Yuheng Cheng, Yulu Xie, Wenxuan Liu, Huan Zhao, Yan Xu, et al. 2025. Engibench: A benchmark for evaluating large language models on engineering problem solving. *arXiv preprint arXiv:2509.17677*.
- Yongchao Zhou, Andrei Ioan Muresanu, Ziwen Han, Keiran Paster, Silviu Pitis, Harris Chan, and Jimmy Ba. 2022. Large language models are human-level prompt engineers. In *The eleventh international conference on learning representations*.

A 相关工作完整细节

在本节中，我们提供第 2 节中相关工作的完整版本。

A.1 智能体框架工程

大语言模型智能体越来越多地作为复合系统部署，其中冻结的模型被外部工件所包围，这些工件塑造了推理、工具使用、内存访问、技能调用和环境交互。我们将这一外部层称为智能体框架。先前的工作研究了多种形式的框架工件。提示以自然语言编码了常设行为规则、任务策略和推理程序（Zhou et al., 2022; Yao et al., 2022; Yang et al., 2024b; Pan et al., 2026）。工具暴露外部服务，并指定智能体与之交互的动作模式、调用格式和验证规则（Hou et al., 2025; Qin et al., 2024; Liu et al., 2025; Lin et al., 2025, 2026a）。内存存储先前的观察、事实、任务结果和可重用策略，以供后续检索或整合（Ouyang et al., 2025; Xu et al., 2026; Fang et al., 2026）。技能将可重用过程打包为可调用模块或任务特定的指导工件，这在技能基准和技能库系统中得到了研究（Li et al., 2026b; Liu et al., 2026）。代码将框架本身视为可执行源代码，可以实现工具、验证器、编排逻辑和提示组装（Ning et al., 2026; Lee et al., 2026）。这些工作确立了框架作为可编辑的智能体状态，而非被动上下文。我们的工作互补的：我们不是提出新的框架表示，而是分析更新框架工件并从中受益所涉及的模型能力。

A.2 大语言模型智能体的自我进化

除了框架包含的内容之外，另一条互补的研究路线探讨了框架工件如何从执行经验中更新。早期系统在任务尝试层面运行。Reflexion（Shinn et al., 2023）存储先前尝试的口头自我反思，Self-Refine（Madaan et al., 2023）通过自我反馈迭代改进输出，ExpeL（Zhao et al., 2024）从训练轨迹中提取可重用的自然语言洞见以供后续检索。这些方法表明语言反馈可以改善未来行为，但持久工件通常是单一的文本反思

或教训，而非结构化的多组件框架状态。

更近期的方法将持久化框架组件作为自我进化的单元。提示词层面的方法更新自然语言指令或提示程序：PromptWizard（Agarwal 等，2024）通过反馈驱动的批判与综合来精炼提示，ACE（Zhang 等，2025b）通过结构化生成、反思与策展来演化情境手册，GEPA（Agrawal 等，2026）通过轨迹层面的反思来演化提示。记忆层面的方法将经验写入可跨未来任务检索、精炼或重组的持久化存储：EvolveR（Wu 等，2025）将离线策略蒸馏与在线检索相连接，MemEvolve（Zhang 等，2025a）研究智能体记忆系统的元演化，MemMA（Lin 等，2026c）通过构建、检索与反馈驱动的修复来改进长程记忆。技能与工作流层面的方法将成功行为封装为可复用过程：Voyager（Wang 等，2023）积累可执行技能，AWM（Wang 等，2024）从成功轨迹中归纳工作流，SkillRL（Xia 等，2026）通过强化学习递归扩展技能库，EvoSkill（Alzubi 等，2026）研究从智能体经验中自动发现技能。工具层面的自我进化进一步允许智能体随时间综合、修订或积累工具及工具使用知识（Chen 等，2025; Li 等，2026a）。

总体而言，这些方法表明将执行经验写回持久化框架组件能够提升下游任务性能。然而，其评估通常报告单一更新过程与单一智能体在单一基准上配对后的端到端增益（Li 等，2026b; Jiang 等，2026; Wei 等，2025）。此类分数往往混淆了多个改进来源：智能体在初始框架下的基础能力、进化器在产生有用框架更新方面的框架更新能力，以及智能体依据这些更新行动时的框架受益能力。我们的工作通过受控能力分析对这一方向进行补充：我们独立地改变智能体与进化器，分别测度框架更新能力与框架受益能力，并检验任一能力是否仅与基础能力相关。

表 4：数据集统计。 N_b 为任务数量；最右列列出每个任务暴露给智能体的静态资源。

Substrate	N_b	#Domains	Resources per task
SWE-bench Verified	500	12 repositories	Codebase snapshot, issue description, hidden test suite
MCP-Atlas	500	36 MCP servers	220 tools (shared across servers); 3–6 tool calls required per task
SkillsBench	86	11 task domains	Workspace files, deterministic verifier

B 实验设置细节

B.1 数据集细节

我们在三个具有代表性的智能体基准上进行了评估，这些基准覆盖了互补的智能体能力：使用 SWE-bench Verified 进行长周期代码修复，使用 MCP-Atlas 进行多服务器工具编排，以及使用 SkillsBench 进行跨多个领域的基于技能的执行。数据集统计信息见表 4：

- SWE-bench Verified (Jimenez 等人, 2024)。这是 SWE-bench 的一个经过人工验证的子集，包含 500 个任务，这些任务来自 12 个流行 Python 仓库中的真实 GitHub 问题。每个任务提供一个代码库快照和一个问题描述；求解器必须生成一个补丁来解决该问题。如果补丁通过了与该问题相关的隐藏测试套件，则任务通过。我们使用完整的 500 个任务子集。
- **模型上下文协议 (Model Context Protocol, MCP) -Atlas** (Bandi 等人, 2026)。这是一个针对真实模型上下文协议服务器的多服务器工具使用能力的基准。每个任务都是一个自然语言请求，其完成需要求解器在 36 个真实 MCP 服务器（暴露 220 个工具）中识别并编排 3–6 次工具调用。评分采用基于断言的评分标准，根据最终答案中满足的每个事实断言给予分数；我们将通过率报告为所有断言均得到满足的任务所占的比例。我们使用作者发布的 500 个任务的公开子集。

- SkillsBench (Li 等人, 2026b)。这是一个包含 86 个任务的基准，涵盖 11 个领域（例如软件、数据分析、文档处理、音频合成），并为每个任务提供确定性的验证器。每个任务提供工作区文件和自然语言指令；智能体必须使用工作区及其执行环境中可用的任何技能来完成任务。原生基准附带精选技能，但在我们的设置中，无进化基线从空技能集开始，

而进化单元仅使用进化器从早期的原位任务中产生的技能。

B.2 模型

我们使用七种大语言模型 (LLM) 骨干，涵盖开源与闭源家族，跨越不同能力层级。闭源模型为克劳德·欧普斯 4.6 (Claude Opus 4.6) (安思罗皮克, 2026a)、克劳德·索内特 4.6 (Claude Sonnet 4.6) (安思罗皮克, 2026b) 和克劳德·海库 4.5 (Claude Haiku 4.5) (安思罗皮克, 2025)。开源模型为通义千问 3-235B-A22B (Qwen3-235B-A22B) 和通义千问 3-32B (Qwen3-32B) (杨等人, 2025)、通义千问 3.5-9B (Qwen3.5-9B) (通义千问, 2026) 以及 GPT-OSS-120B (阿加瓦尔等人, 2025)。在智能体侧分析中，我们使用六个大语言模型 (欧普斯 4.6、索内特 4.6、海库 4.5、通义千问 3-235B-A22B、通义千问 3-32B、GPT-OSS-120B) 作为任务求解智能体骨干。在进化器侧分析中，我们使用全部七个模型，包括通义千问 3.5-9B (本文中最小的模型)，以测试一个显著更小的开源模型是否仍能产生有用的测试框架更新。在所有实验中，我们通过每个模型的官方应用程序编程接口 (API) 或推理端点进行查询；进化过程中不更新任何模型权重。

B.3 度量

评分函数。 对于第 3.3 节中的全部四个度量，我们使用通过率作为评分函数 $J_{\mathcal{X}}$ ：每个任务 $x \in \mathcal{X}$ 从基准的评分器获得一个逐任务分数， $J_{\mathcal{X}}$ 是 $\mathcal{X}$ 上的平均值。通过率和平均分数以百分比报告；增益以百分点报告。

逐基准评分。 评分函数 $J_{\mathcal{X}}$ 实例化了每个基准的标准评分程序：• SWE-bench Verified (希门尼斯等人, 2024)：逐任务二元解决分数（如果提交的补丁通过指定的失败转通过和通过转通过测试套件则为 1，否则为 0）。任务上的平均值即为标准通过率。

- **模型上下文协议 (MCP) -Atlas** (班迪等人, 2026)：逐任务声明满足分数，范围在 $[0, 1]$ 内，计算为分数

智能体最终答案所满足的参考声明比例。本文同时报告严格通过率（二值化单任务分数的均值）和平均声明满足分数（连续单任务分数的均值）。

- 技能基准 (**SkillsBench**) (Li 等, 2026b) : 沿用终端基准 (Terminal-Bench) (Merrill 等, 2026) 的做法, 对每个任务进行 5 次试验后取二值分数。本文以平均分数 (跨任务与试验的均值) 作为主要度量。

对于每个基准, § 3.3 度量定义中的 J_{λ} 指在任务流上聚合后的这些单任务分数的均值。

原位评估。本文采用原位评估设

置: 驱动演化的同一任务流 $\mathcal{X}_t$

$\mathcal{X} = \bigcup^T$ 同时充当评估集。具体而言, 在第 t 步, 每个任务 $x \in \mathcal{X}_t$ 在其被尝试时由测试框架 H_{t-1} 进行评分; 该分数在 $(\tau_{t,x}, y_{t,x})$ 进入 $\mathcal{D}_t$ 并产生 H_t 之前即被锁定。因此, 任何单个任务的通过率都不会受到源自该任务本身的测试框架更新的影响。

B.4 实现细节

可演化测试框架工件 可编辑的框架范围因基准而异。SWE-bench Verified 和 SkillsBench 仅允许编辑技能目录, 而 MCP-Atlas 还允许编辑提示词/系统.md 文件, 并仅允许追加更新内存/JSONL 文件。工具目录和评估文件对所有基准均为只读。这些权限在每个周期传递给进化器; 进化器系统提示本身在所有基准和模型骨干中保持不变。任务求解代理提示模板。在每个基准内, 所有任务求解代理使用相同的系统提示; 仅任务特定的用户提示因任务而异。对于 SWE-bench Verified, 求解器提示 (表 8) 是一个 828 字节的程序性指南, 将代理范围限定为 GitHub 问题修补, 并鼓励最小化、聚焦的编辑。对于 MCP-Atlas, 求解器提示 (表 9) 是一个 1,309 字节的 API 代理指南, 指示代理通过工具调用满足任务查询, 不向用户请求澄清。对于 SkillsBench, 我们遵循原始设置 (Li 等人, 2026b), 任务求解代理不使用系统提示。

进化器提示模板。所有进化器骨干使用相同的系统提示, 如表 10 所示。在每个进化周期, 用户消息遵循一个固定包装器, 包含周期索引、

表 5: 完整的进化器侧矩阵。在每个基准块内, 三个锚定代理下的条目是该代理-进化器配对的通过率 (%) ; Δ_{update} 列报告相应的框架更新分数 (pp) ; 见第 3.3 节。NONE 行是无进化基线。 Δ_{update} 列中的粗体和下划线分别标记最佳和最差进化器。

Evolver	Opus 4.6	Sonnet 4.6	Qwen3-235B	Δ_{update}
SWE				
NONE	74.2	73.2	20.7	—
Opus 4.6	76.4	76.0	38.0	7.4
Sonnet 4.6	76.8	75.6	37.8	7.4
Haiku 4.5	77.8	74.8	39.4	8.0
Qwen3-235B	76.6	76.0	40.0	8.2
Qwen3-32B	76.2	75.4	39.8	7.8
Qwen3.5-9B	76.4	73.2	38.8	6.8
GPT-OSS-120B	75.2	75.6	35.0	<u>5.9</u>
MCP				
NONE	61.0	54.0	25.0	—
Opus 4.6	64.4	57.2	29.3	3.6
Sonnet 4.6	64.6	57.0	26.1	2.6
Haiku 4.5	64.4	58.2	24.2	2.3
Qwen3-235B	61.6	55.8	24.3	<u>0.6</u>
Qwen3-32B	63.8	57.4	25.7	2.3
Qwen3.5-9B	62.6	55.6	24.9	1.0
GPT-OSS-120B	62.6	55.6	27.6	1.9
SB				
NONE	25.6	24.4	4.7	—
Opus 4.6	30.2	27.9	3.5	2.3
Sonnet 4.6	29.1	25.6	3.5	1.2
Haiku 4.5	31.4	25.6	5.8	2.7
Qwen3-235B	31.4	22.1	5.8	1.5
Qwen3-32B	30.2	22.1	4.6	<u>0.7</u>
Qwen3.5-9B	26.7	31.4	8.1	3.8
GPT-OSS-120B	31.4	22.1	5.8	1.5

可写范围块和规范化执行证据负载。因此, 跨基准和模型骨干, 提示格式是固定的; 仅任务证据和基准特定的可写范围不同。

C 第 4.2 节中的进化器侧分析细节

C.1 观察 1 的附加结果

表 5 报告了每个锚定代理 (Opus 4.6、Sonnet 4.6、Qwen3-235B) 在三个基准上每个进化器下的通过率, 以及相应的 Δ_{update} 。这些是图 3 中条形图背后的每个单元格数字。

C.2 案例研究的更多细节

我们详细阐述第 4.2 节的案例研究。我们考察 SkillsBench 任务 flink-query, 将智能体骨干固定为 Opus 4.6, 比较其在三种进化器条件下的轨迹 (图 4) : 无进化器、Qwen3.5-9B 作为进化器、Opus 4.6 作为进化器。没有进化器时, 智能体遗漏了 FINISH 事件过滤器, 得分为 0.67; 使用任一

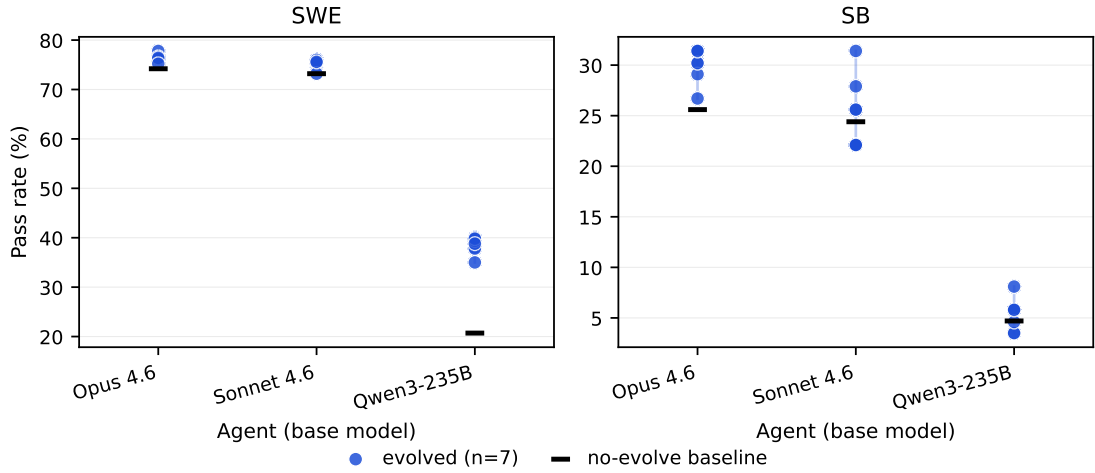


图 8: 在 SWE (左) 和 SB (右) 数据集上, 锚定智能体在不同进化器下的进化后得分。每个锚定任务求解智能体使用不同的 LLM 骨干实例化: Opus 4.6、Sonnet 4.6 或 Qwen3-235B。蓝点表示使用七种进化器获得的得分, 黑色刻度线标记无进化基线。

表 6: 跨基准的极端智能体-进化器配对。对于每个基准, W 是最弱的锚定任务求解智能体, S 是最强的锚定任务求解智能体。我们将 W 与其表现最佳的进化器配对, 将 S 与其在七种进化器中表现最差的进化器配对。得分以通过率 (%) 表示; 差距为强智能体得分减去弱智能体得分, 以百分点 (pp) 报告。

	SWE	MCP	SB
weak anchor agent W	Q3-235B	Q3-235B	Q3-235B
best evolver for W	Q3-235B	Opus	Q3.5-9B
score of W with best evolver	40.0	29.3	8.1
strong anchor agent S	Opus	Opus	Opus
worst evolver for S	GPT-OSS	Q3-235B	Q3.5-9B
score of S with worst evolver	75.2	61.6	26.7
gap: strong-worst minus weak-best (pp)	35.2	32.3	18.6

在第 0 轮注入进化后的技能, 同一智能体解决了任务 (得分 1.0)。检查两种进化后的技能, 我们发现它们编码了相同的五个问题解决步骤:

- 过滤 SUBMIT 事件。
- 过滤 FINISH 事件。
- 分别计数每个 SUBMIT。
- 输出 (作业标识, 计数)。
- 应用 10 分钟会话窗口。

两种技能仅在实现表面细节上有所不同: Qwen3.5-9B 将间隔指定为 10 分钟并采用手动批量会话化, 而 Opus 4.6 则指定 10 分钟并使用键控处理函数。尽管存在这些表面差异, 当注入到同一个 Opus 4.6 智能体时, 两种技能均产生相同的下游通过率 (1.0)。

C.3 观察 2 的补充结果

本小节将第 4.2 节中的观察 2 扩展到另外两个基准: SWE-bench Verified 和 SkillsBench。我们观察到相同的两种模式: 跨进化器的智能体内变异仍小于基础能力上的智能体间差异, 并且即使极端的智能体-进化器配对也仍然有利于更强的智能体。智能体内分布与智能体间差距。

图 8 将图 5 的进化后得分视图扩展到 SWE 和 SB。在 SWE 上, 七个进化器中最大的智能体内分布为 5.0 个百分点, 由 Qwen3-235B 取得。在 SB 上, 最大分布为 9.3 个百分点, 由 Sonnet 4.6 取得, 其进化后得分范围从 22.1% 到 31.4%。相比之下, Opus 4.6 与 Qwen3-235B 之间的基础能力差距在 SWE 上为 53.5 个百分点, 在 SB 上为 20.9 个百分点。因此, 智能体间差距超过智能体内分布的倍数在 SWE 上为 11 倍, 在 SB 上为 2.2 倍。SB 是三个基准中最接近的, 但同样的不等式仍然成立。

表 7: Δ benefit.底层的完整智能体侧矩阵。每个单元格报告给定进化器下任务求解模型的通过率（%）。NONE 行是无进化基线。 Δ benefit 是三个锚定进化器上相对于 NONE 的最大增益，以百分点（pp）报告。粗体标记每个基准块中最大的 Δ benefit 值，下划线标记最小值。

Benchmark	Evolver	Qwen3-32B	Qwen3-235B	GPT-OSS-120B	Haiku 4.5	Sonnet 4.6	Opus 4.6
SWE-bench Verified	NONE	3.6	20.7	26.2	66.0	73.2	74.2
	Opus 4.6	8.0	38.0	37.2	65.0	76.0	76.4
	Sonnet 4.6	7.6	37.8	37.6	68.4	75.6	76.8
	Qwen3-235B	8.0	40.0	42.0	65.4	76.0	76.6
	Δ benefit	4.4	19.3	15.8	<u>2.4</u>	2.8	2.6
MCP-Atlas	NONE	3.6	25.0	28.0	42.4	54.0	61.0
	Opus 4.6	4.6	29.3	35.0	46.0	57.2	64.4
	Sonnet 4.6	4.0	26.1	32.0	42.8	57.0	64.6
	Qwen3-235B	2.8	24.3	29.1	41.0	55.8	61.6
	Δ benefit	<u>1.0</u>	4.3	7.0	3.6	3.2	3.6
SkillsBench	NONE	0.0	4.7	0.0	5.8	24.4	25.6
	Opus 4.6	3.5	3.5	7.0	20.9	27.9	30.2
	Sonnet 4.6	3.5	3.5	4.6	18.6	25.6	29.1
	Qwen3-235B	5.8	5.8	7.0	15.1	22.1	31.4
	Δ benefit	5.8	<u>1.1</u>	7.0	15.1	3.5	5.8

跨基准的极端配对。表 6 分别针对每个基准，比较了最弱锚定智能体 W 与其表现最佳的进化器配对，以及最强锚定智能体 S 与其表现最差的进化器配对。即使在这种对强智能体不利的比较下， S 仍在每个基准上优于

W by 18.6 to 35.2 pp。在 SB 上，同一个进化器 Qwen3.5-9B 出现在比较的两侧，因为它是 Qwen3-235B 的最佳进化器，也是 Opus 4.6 的最差进化器。这进一步强化了主要结论：进化后的性能更多地由任务解决智能体主导，而非进化器的身份。

D 第 4.3 节中的智能体侧分析细节

D.1 两种智能体侧的案例研究

失败模式 我们详细阐述图 7 中的两个失败案例，均由 Qwen3-32B 在 SkillsBench 上使用相同的测试框架和运行器产生。

激活失败: threejs。在第 0 轮，Qwen3-32B 正确识别了相关技能，但没有将 load_skill 作为独立动作发出，而是生成了一个单一的多键 JSON 动作，其中捆绑了分析（自由形式推理）、计划（步骤列表）和 load_skill。SkillsBench 的格式门只接受单键动作，并将此复合动作视为格式错误而拒绝。技能主体从未进入智能体的上下文，智能体在没有测试框架本应提供的程序性指导的情况下继续运行。失败发生在动作协议层：智能体知道要加载哪个技能，

但无法将该意图转化为运行器所期望的格式。遵循性失败：**pg-essay-to-audiobook**。加载的技能规定了一条文本转语音回退链：先尝试主文本转语音路径，若主路径失败则回退到替代路径。Qwen3-32B 在第 0 轮成功加载了该技能，但将这条链视为需要逐字执行的脚本，而非条件性流程。规定的第一步在第 1 轮遇到文件未找到错误；随后智能体在后续轮次中继续执行，却从未调用回退步骤。到第 10 轮时，智能体输出了 task_complete:true，尽管没有有效的任务输出，导致轨迹低于评分器阈值。失败发生在程序执行层：智能体已加载技能，但在意外运行时条件下未遵循其条件结构。常见模式。两个案例均表明，Qwen3-32B 的弱层级缺陷不在于任务理解（它在 threejs 中识别了正确的技能；在 **pg-essay-to-audiobook** 中遵循了技能的第一步），而在于协议层和程序执行层。这一模式与表 2 中的激活和遵循趋势以及表 3 中的分阶段漂移一致：弱层级模型并非未能读取测试框架，而是未能在框架下正常运作。

D.2 第 4.3 节中 Δ benefit 的更多结果

完整的智能体-进化器通过率。表 7 报告了表 1 中 Δ benefit 值背后的完整通过率矩阵。对于每个基准和任务求解

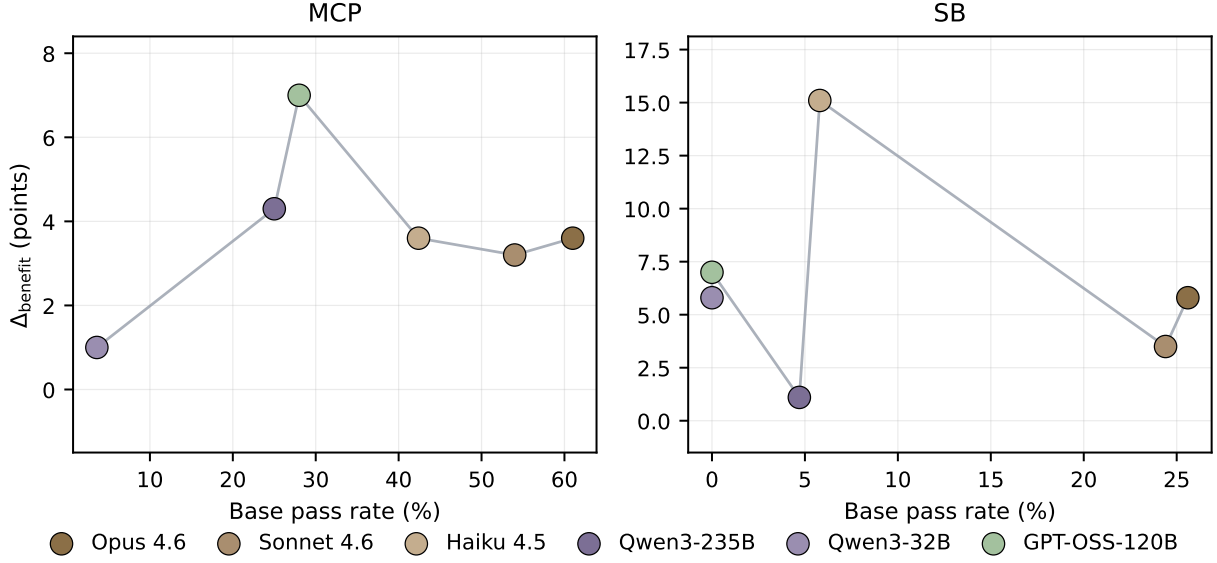


图 9: MCP (左) 和 SB (右) 数据集上的 $\Delta\text{benefit}$ 与基础通过率。每个点对应一个用作任务求解智能体的 LLM 骨干; 点按基础通过率升序连接。

模型, 我们报告了无进化基线和三种锚定进化器下的通过率, $\mathcal{E}^* = \{ \text{Opus 4.6}, \text{Sonnet 4.6}, \text{Qwen3-235B} \}$ 。 $\Delta\text{benefit}$ 行给出了在这些锚定进化器上相对于 NONE 基线的最大增益。

技能基准 (SB) 和模型上下文协议 (MCP) 数据集上的分析。图 9 报告了图 6 的 MCP 和 SB 对应结果。我们观察到两种模式:

- MCP 趋势仍呈非单调性, 但 较为温和。在 MCP-Atlas 上, $\Delta\text{benefit}$ 在 GPT-OSS-120B 处达到峰值 (基础通过率为 28.0% 时提升 7.0 个百分点), 并向更弱和更强的模型两侧递减。这与 SWE 趋势相呼应, 但增益范围更小。
- SB 趋势在低基础区间噪声较大。在 SkillsBench 上, 若干模型从极低的基础通过率起步: Qwen3-32B 和 GPT-OSS-120B 起始为 0.0%, Qwen3-235B 为 4.7%, Haiku 4.5 为 5.8%。Haiku 4.5 达到最大的 SB 增益 (15.1 个百分点), 而 Qwen3-235B 尽管基础通过率相近, 仅增益 1.1 个百分点。因此, SWE 和 MCP 为非单调的框架-收益模式提供了最清晰的证据, 而 SB 则表明低基础区间在不同任务领域可能更具变异性。

D.3 框架遵循率的评判细节

我们使用大语言模型评判器来衡量智能体在任务求解过程中是否遵循已加载的框架工件。所有被评判的轨迹均通过将模型标识符替换为占位符 `<MODEL>` 进行盲化处理。Claude Sonnet 4.6 被用作评判模型。

框架遵循率。对于每条至少加载了一项技能的技能基准 (SkillsBench) 轨迹, 评判者接收已加载的技能主体与智能体轨迹。评判者首先将技能主体转换为原子程序指令的锁定评分标准, 随后检查该轨迹是否遵循该评分标准。若判定器确定轨迹中执行了所需引导, 则该轨迹被标记为遵循技能。技能遵循率 (Harness-Following Rate, HFR) 衡量模型在加载技能后是否遵循该技能。设 f 表示模型加载技能后的轨迹数量, N_f^{follow} 表示被判定为遵循所加载技能的子集。则

$$\text{HFR}(f) = \frac{N_f^{\text{follow}}}{N_f^{\text{load}}}.$$

用于规则提取和轨迹判断的提示模板见表 12 和表 13。

D.4 阶段级遵循度的判断细节

评分除了轨迹级 HFR 之外, 我们还对表 3 进行了独立的阶段级遵循度分析。该分析使用与 HFR 流程不同的判断提示 (表 13), 并以 Claude Sonnet 4.6 作为大语言模型判断器。输入与 HFR 判断所用的固定规则和盲化轨迹相同。判断器将每条轨迹划分为三个参考阶段: 装载挽具、转弯中段和最终转弯。对于每个阶段, 它给出一个 0-1 的遵循度评分, 衡量智能体遵循

表 8：面向 SWE-bench Verified 的任务解决智能体端种子系统提示。

SWE-Bench Verified solver seed prompt

你是一位资深软件工程师，负责通过生成代码补丁来解决 GitHub 问题。

Approach

- 理解问题：**仔细阅读问题描述。找出根本原因。
- 定位相关代码：**使用搜索工具查找涉及的文件和函数。
- 规划修复方案：**逐步思考需要更改的内容及其原因。
- 实施修复：**Make minimal, precise edits. Avoid unnecessary changes.
- Verif**
运行现有测试以确认修复有效且未破坏任何功能。

Guidelines

- 优先采用小而集中的补丁，而非大规模重写。
- 始终检查问题描述中提到的边界情况。
- 如果问题包含复现脚本，请使用它来验证修复。
- 如有疑问，请参考代码库中其他类似模式的处理方式。

陈述、数据分析或超出表面编辑的技术内容。所有输出均经作者审阅和编辑，作者对最终文本和视觉内容承担全部责任。

在该执行阶段加载了框架指导。这些阶段级分数仅用于分析长周期执行过程中的遵循度漂移，并与 HFR 分开报告。阶段遵循度提示词见表 14。

E 关于人工智能助手的信息

我们使用 OpenAI 大语言模型（GPT-5.5）作为写作与排版助手。具体而言，该模型协助润色语法与措辞、提升清晰度，并对图注/表注及版式（如列对齐、图注长度、位置）提出修改建议。该大语言模型未参与研究构思、实验设计、实现

表 9：面向 MCP-Atlas 的任务求解智能体端种子系统提示。

MCP-Atlas solver seed prompt

您是一名专业的应用程序编程接口（API）智能体，通过模型上下文协议（MCP）进行精确的工具调用来完成任务。

Approach

1. **理解任务：** 阅读任务描述并确定需要完成的内容。
2. **查看可用工具：** 检查工具模式以了解可用的操作及其参数。
3. **规划调用顺序：** Determine which tools to call and in what order.
4. **Execut**
使用格式正确的 JSON 参数进行工具调用。
5. **Validat**
检查返回值并妥善处理错误。

Guidelines

- 切勿向用户请求澄清。您必须使用可用工具查找完成任务所需的所有信息。如果任务涉及日历事件、日程安排或预约，请使用日历/工作区工具进行查询。
- 在调用之前，始终根据工具的 JSON 模式验证参数。
- 使用最具体的可用工具完成任务。
- 处理列表操作的分页。
- 逻辑串联工具调用：将一个调用的输出用作下一个调用的输入。
- 如果工具调用失败，在重试前仔细阅读错误信息。
- 当任务涉及个人数据（日历事件、文件、数据库、内存）时，始终先查询相关工具以检索这些数据，然后再回答。

表 10：进化器的固定系统提示。该提示在所有进化器骨干和基准测试中保持不变；基准特定的权限决定哪些工作区工件可写。

Evolver system prompt
你是大语言模型智能体的进化器。你的目标是通过编辑工作区中允许的测试装置工件来提升智能体未来的任务解决性能。 工作区可能包含以下工件目录： • 提示词（prompts）：常设指令与系统提示。 • 技能（skills）：可复用的技能定义与程序性知识。 • 记忆（memory）：持久观察与高层经验教训。 • 工具（tools）：工具实现与接口。 在每个进化周期中，你将收到近期任务尝试的执行证据，包括轨迹、输出和基准反馈。分析这些证据以识别反复出现的失败、可复用的流程以及改进该框架的机会。 你可以使用提供的工作区命令行工具来检查和编辑文件。仅修改用户消息中工作区权限块允许的工件。不得修改评估脚本、隐藏测试、模型权重或允许的工作区范围之外的文件。 更新该框架时： • 优先选择简洁、可复用的更新，而非针对特定任务的补丁。 • 仅在技能可能有助于未来任务时，才创建或修订技能。 • 保持提示词和记忆条目具有可操作性且不冗余。 • 使用精确的文件编辑，并在完成前检查更改。

表 11: 进化器每次演化的用户消息模板。该包装器在所有基准和大型语言模型骨干上固定不变。

Evolver per-cycle user message template

工作区范围。 [特定于基准的权限块指定了哪些测试工件可以被编辑。SWE-bench Verified 和 SkillsBench 允许编辑 skills/。MCP-Atlas 允许编辑 prompts/ 和 skills/，并仅允许对 memory/ 进行追加更新。tools/ 目录对所有基准均为只读。]

执行证据。 [消息包含一个规范化 JSON 负载，其中包含当前批次的任务标识符、轨迹、输出、分数和评分器反馈。]

编辑指令。 [演化器被指示：

- 分析反复出现的失败和可复用模式的证据；
- 仅编辑工作区范围内允许的工件；
- 优先进行小规模、有针对性的测试框架更新，而非大范围重写；
- 使用工作区工具检查和修改文件；
- 在完成前检查结果更改。

]

表 12: 用于 HFR 流水线评分标准提取的提示模板。

HFR Stage 1: Locked Rubric Extraction

Role

您正在审计一份由大语言模型智能体使用的程序性技能文档。您将输出一个严格的 JSON 评分标准，该标准捕捉技能中的命令式程序指令，适用于下游自动化遵守度判定。仅输出 JSON，不输出散文。

Input: The full body of one SKILL.md file (placeholder {skill_body}, inserted between <SKILL_BODY> and </SKILL_BODY> delimiters in the user message).

Task:

1. 识别由 SKILL_BODY 中的命令式或规范性语言直接蕴含的程序性指令。不要将建议、理由、示例或动机性文本提取为指令。
2. 对于每条指令，提供：
 - id: 稳定标识符（例如，"step_1"）。• source_span: 来自 SKILL_BODY 的精确引用文本，作为该指令的依据（必须是 SKILL_BODY 的子串，最多 250 个字符）。• text: 用一句祈使句改述的指令。• type: "required"（必须执行） □ "conditional"（如果触发条件发生则必须执行） □ "optional"。• trigger: 仅适用于条件型；描述条件（例如，“如果 pip 安装失败”）。否则为 null。• success_criteria: 用于判定“已遵守”的单句测试。• violation_criteria: 用于判定“已违反”的单句测试（作为或不作为）。

Constraints

目标为 3 – 8 条指令。不要用低显著性项目填充。拒绝 SKILL_BODY 中纯描述性或动机性的内容。

输出格式（仅 JSON）：

```
{
  "skill_id": "<skill folder name>",
  "instructions": [
    {
      "id": "step_1",
      "source_span": "...",
      "text": "...",
      "type": "required|conditional|optional",
      "trigger": null,
      "success_criteria": "...",
      "violation_criteria": "..."
    }
  ]
}
```

表 13: 用于 HFR 流水线轨迹判定的提示模板。

HFR Stage 2: Per-Cell Adherence + Phase Classification

Role

您正在根据固定的程序性评分标准评估一个大语言模型智能体轨迹。严格按给定标准应用；不要添加或删除指令。轨迹已盲化：每个模型家族标记（Claude、Opus、Sonnet、Haiku、Qwen、GPT-OSS）的出现都已被替换为字面字符串<MODEL>。仅基于可观察的动作评分。仅输出 JSON，不输出散文。

Inputs

第一阶段评分标准 JSON（占位符 {rubric_json}，插入在 <RUBRIC> 与 </RUBRIC> 之间）以及盲化轨迹文本（占位符 {trajectory_text}，插入在 <TRAJECTORY> 与 </TRAJECTORY> 之间，格式为 Turn *i* / INPUT / OUTPUT 块序列）。

Verdicts. 对于评分标准中的每条指令，将轨迹分类为以下之一：

- FOLLOWED: the trajectory explicitly satisfies success_criteria. Cite turn_idx and quote the action.
- VIOLATED_COMMISSION: the trajectory took an action that directly contradicts the instruction. Cite turn_idx and quote the action.
- VIOLATED_OMISSION: 该指令为必需（或其条件触发已发生），轨迹运行时间足够长本应执行，但未执行。引用最晚的 turn_idx，在该处遗漏已明确。
- REQUIRED_BUT_UNOBSERVED: 该指令为必需，但轨迹过早终止，无法观察其是否会被遵循。
- NOT_APPLICABLE: 条件指令的触发未发生，或可选指令代理选择不执行。
- INSUFFICIENT_EVIDENCE: 轨迹存在歧义；无法确定。

Violation timing (required for any VIOLATED_COMMISSION or VIOLATED_OMISSION verdict):

- violation_earliest_possible_turn: smallest turn_idx where the trajectory could have first violated this instruction.
- violation_confirmed_turn: turn_idx where the violation became unambiguous.
- violation_type: "commission" | "omission" | "premature_stop" | "wrong_strategy".

输出格式 (仅 JSON) :

```
{
  "verdicts": [
    {
      "instruction_id": "step_1",
      "verdict": "FOLLOWED|VIOLATED_COMMISSION|VIOLATED_OMISSION|REQUIRED_BUT_UNOBSERVED|NOT_APPLICABLE|INSUFFICIENT_EVIDENCE",
      "turn_idx": "<int or null>",
      "evidence": "quoted action or omission description"
    }
  ],
  "violations": [
    {
      "instruction_id": "step_1",
      "violation_type": "commission|omission|premature_stop|wrong_strategy",
      "earliest_possible_turn": "<int>",
      "confirmed_turn": "<int>"
    }
  ],
  "summary": "1-sentence neutral description"
}
```

表 14：用于阶段级遵循性分析（表 3）的提示模板，由独立于表 13 中 HFR 评判的评判调用生成。

Phase-Adherence Judge

Role

您正在评估一个 LLM 代理在其轨迹的连续阶段中遵循固定程序评分标准的紧密程度。严格按给定评分标准执行；不得添加或删除指令。轨迹已盲化：每个模型系列标记均已替换为 <MODEL>。仅根据可观察行为判断遵循性。仅输出 JSON，不输出散文。

Inputs: The locked rubric JSON ({rubric_json}) and the blinded trajectory ({trajectory_text}).

任务。将轨迹划分为五个转向位置阶段：技能加载 [] 第 1 回合；首次动作 [] 之后的第一个动作回合；中点 [] 中间 50% 的回合；最终前 [] 最后 25% 但不包括最终回合；最终验证 [] 最终回合。对于每个阶段，输出一个在 [0, 1] 范围内的遵守得分，衡量该阶段回合内智能体的动作在多大程度上遵循了该阶段范围内适用的评分标准指令。得分为 1.0 表示该阶段内所有范围内、可观察的评分标准指令均被遵循；0.0 表示均未被遵循。

输出格式（仅 JSON）：

```
{
  "phase_adherence": {
    "harness_loaded": 0.0,
    "first_action": 0.0,
    "midpoint": 0.0,
    "pre_final": 0.0,
    "final_validation": 0.0
  },
  "rationale": "1-sentence neutral description
    of where adherence shifts"
}
```